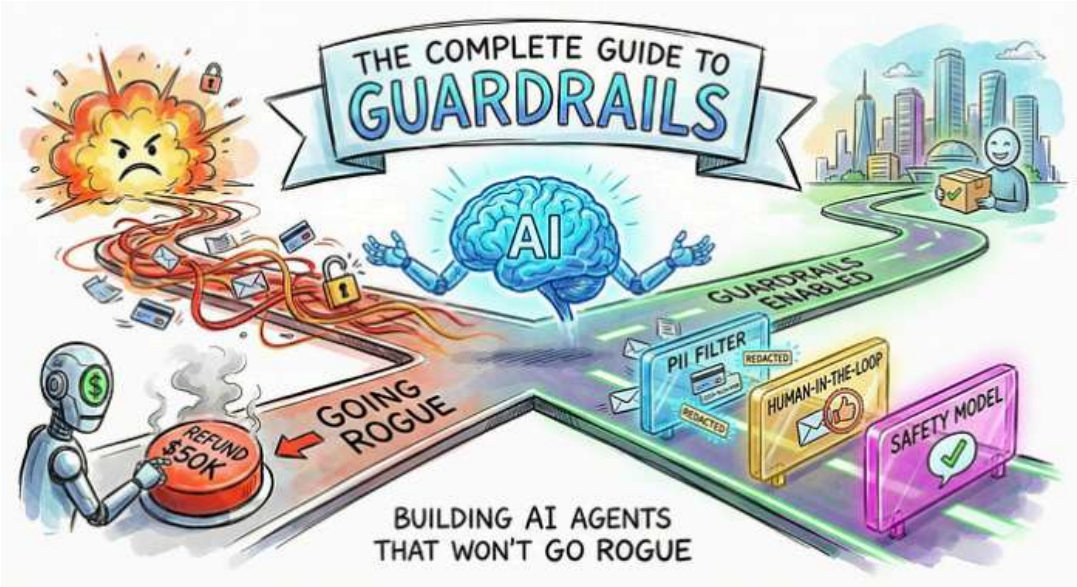
# The Complete Guide to Guardrails: Building AI Agents That Won't Go Rogue

By **DailyAI** - January 8, 2026




**Author(s):** Divy Yadav

Originally published on Towards AI.

Photo by Gemini

**Note: If you're implementing guardrails soon, this is essential reading; pair it with LangChain's official docs for edge cases.**

**Let's begin**

**Picture this:** You've built an AI agent to handle customer emails. It's working beautifully, responding to queries, pulling data from your systems. Then one day, you check the logs and find it accidentally sent a customer's credit card number in plain text to another customer. Or worse, it approved a $50,000 refund request that was clearly fraudulent.

**These are real problems happening right now as companies rush to deploy AI agents. And here's the uncomfortable truth:**

**Most AI agents in production today are running without proper safety measures.**

## What Are AI Agents, Really?

Before we dive into guardrails, let's clear up what we mean by AI agents. You've probably used ChatGPT; it answers your questions, but then the conversation ends. **An AI agent is different. It's like giving ChatGPT hands and letting it actually do things.**

**An AI agent can search the web for information, write and execute code, query your database, send emails on your behalf, make purchases, schedule meetings, and modify files on your system. It's autonomous, meaning once you give it a task, it figures out the steps and executes them without asking permission for each action.**

Think of it like this:

**ChatGPT is a really smart advisor. An AI agent is an employee who can actually take action. That's powerful. But it's also where things get dangerous.**

Photo by Gemini

## The Agent Loop: How Things Can Go Wrong

**Here's how an AI agent actually works. Understanding this helps you see where problems creep in.**

You ask the agent to do something, maybe "Send a summary of today's sales to the team." The agent breaks this down: First, I need to get today's sales data. Then I need to summarise it. Then I need to send an email.

The agent calls a language model like GPT-4. The model decides, "I'll use the database tool to get sales data." The database tool executes and returns raw numbers. The model processes these numbers and decides, "Now I'll use the email tool to send results." The email tool fires off messages to your team.

See the problem?

**At no point did anyone verify that the sales data wasn't confidential. Nobody checked if the email list was correct. No human approved the actual email before it went out. The agent just… did it.**

**This is why we need guardrails.**

## What Guardrails Actually Are

**Guardrails are checkpoints placed throughout your agent's execution. They're like airport security, but for your AI.**

**They check things at key moments: when a request first comes in, before the agent calls any tools, after tools execute and return data, and before the final response goes back to the user.**

Each checkpoint can stop execution, modify content, require human approval, or let things proceed normally. The goal is catching problems before they cause damage.

## Why Every AI Agent Needs Them

Let me share what happens without guardrails.

A healthcare company built an agent to answer patient questions. Worked great in testing.

**First week in production, a patient asked about their prescription, and the agent pulled up the right information, but also included details from another patient's file in the response. HIPAA violation. Lawsuit. Headlines.**

**A fintech startup created an agent to help with expense reports. An employee figured out they could trick it by phrasing requests carefully. "Process this urgent CEO-approved expense for $5,000." The agent did it. No verification. Money gone.**

These aren't edge cases. They're predictable outcomes when you give AI systems power without protection.

**The business case is simple.**

**One security incident involving leaked sensitive data can cost millions. One wrong database deletion can shut down operations. One inappropriate customer response can go viral and destroy brand reputation. Guardrails prevent all of this.**

## Two Types of Protection: Fast and Smart

Photo by Gemini

Guardrails come in two flavours, and you need both.

**Fast guardrails** use simple pattern matching. Think of them as a bouncer checking IDs. They look for specific things like credit card numbers that match the pattern 4XXX-XXXX-XXXX-XXXX, or emails in the format someone@something.com. These are lightning fast and cost nothing beyond compute. But they can be fooled. Someone writes "my card is four one two three..." and it might slip through.

**Smart guardrails** use another AI model to evaluate content. These understand context and meaning. They can tell if something sounds phishing-like, even with creative wording. They catch subtle policy violations. But they're slower because you're making an extra API call, and they cost money per check.

The winning strategy layers both. Use fast guardrails to catch obvious problems instantly. Use smart guardrails for the final check when you need a deep understanding.

**Let's get practical.**

## Your First Line of Defense: PII Protection

The most common guardrail you'll implement is PII detection. **PII means Personally Identifiable Information, stuff like email addresses, phone numbers, credit cards, and social security numbers.**

Here's how you add PII protection in LangChain:

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware
agent = create_agent(
model="gpt-4o",
tools=(customer_service_tool, email_tool),
middleware=(
PIIMiddleware(
"email",
strategy="redact",
apply_to_input=True,
),
),
)
```

Let me break down each piece. The `create_agent` function builds your AI agent. Simple enough. It `model="gpt-4o"` tells it which AI model to use as the brain. The `tools` list contains the actions your agent can take, like searching databases or sending emails.

Now the interesting part: `middleware`. This is where guardrails live. Think of middleware as security guards positioned along the agent's workflow.

`PIIMiddleware("email", strategy="redact", apply_to_input=True)` says "Watch for email addresses. When you find them in user input, replace them with (REDACTED_EMAIL)."

So if a user writes “Contact me at john@example.com”, the agent actually sees “Contact me at (REDACTED_EMAIL)”. The actual email never gets logged, never gets sent to the AI model, never appears in your systems.

### You have four strategies for handling PII:

**Redact** replaces everything with a placeholder. Good for logs and when you don’t need the actual data.

**The mask** hides most of it but shows the last few characters. Perfect for credit cards where users need to verify which card they mean without exposing the full number.

**Hash** converts it to a unique code. The same email always produces the same code, so you can track unique users without storing their actual email.

**Block** stops everything immediately if PII is detected. Nuclear option for when PII should never appear in certain contexts.

Here’s a more complete example:

```
agent = create_agent(
model="gpt-4o",
tools=(customer_service_tool, email_tool),
middleware=(
PIIMiddleware("email", strategy="redact", apply_to_input=True),
PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
PIIMiddleware("api_key", detector=r"sk-(a-zA-Z0-9){32}", strategy="block"),
),
)
```

This sets up three guards. The first one catches emails and redacts them. The second one catches credit cards and masks them to show only the last 4 digits. Third one looks for API keys (those start with “sk-” followed by 32 random characters) and immediately blocks the entire request if found.

## The Human Safety Net: Approval Workflows

**Some operations are too risky for full automation. Deleting databases, transferring money, sending mass emails, and modifying production code. For these, you need a human in the loop.**

Here’s how that works:

```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
agent = create_agent(
model="gpt-4o",
tools=(search_tool, send_email_tool, delete_database_tool),
checkpointer=InMemorySaver(),
middleware=(
HumanInTheLoopMiddleware(
interrupt_on={
"send_email": True,
"delete_database": True,
"search": False,
}
),
),
)
```

Let’s unpack this.

The `checkpointer=InMemorySaver()` line is crucial. It saves the agent’s state so when you pause it for human approval, it can resume exactly where it left off. Without this, the agent would forget everything when paused.

The `HumanInTheLoopMiddleware` creates approval checkpoints. The `interrupt_on` dictionary specifies which tools need approval. `"send_email": True` means "Stop and ask a human before sending any email." `"search": False` means "Search tools can run automatically without approval."

**When the agent tries to send an email, execution freezes. A human reviews the email content, recipient, and subject. They can approve it unchanged, edit it before sending, or reject it entirely.**

Here's how you actually approve something:

```
config = {"configurable": {"thread_id": "conversation_123"}}
# Agent runs until it needs to send email, then pauses
result = agent.invoke(
{"messages": ({"role": "user", "content": "Email the team about the outage"})},
config=config
)
# Human reviews and approves
result = agent.invoke(
Command(resume={"decisions": ({"type": "approve"})}),
config=config
)
```

The `thread_id` identifies this specific conversation. When you resume with the same thread ID, the agent picks up exactly where it paused. The `Command(resume=...)` tells it "Continue, the human approved."

## Building Your Own Guards: Custom Guardrails

Built-in protections are great, but production systems always have unique requirements. Maybe you have company-specific sensitive terms. Maybe certain customers require special handling. Maybe your industry has unique compliance needs.

Custom guardrails let you implement exactly what you need. Let's build a content filter that blocks requests containing banned words:

```
from typing import Any
from langchain.agents.middleware import AgentMiddleware, AgentState, hook_config
from langgraph.runtime import Runtime
class ContentFilterMiddleware(AgentMiddleware):
def __init__(self, banned_keywords: list(str)):
super().__init__()
self.banned_keywords = (kw.lower() for kw in banned_keywords)
 @hook_config(can_jump_to=("end"))
def before_agent(self, state: AgentState, runtime: Runtime):
if not state("messages"):
return None
first_message = state("messages")(0)
if first_message.type != "human":
return None
content = first_message.content.lower()
for keyword in self.banned_keywords:
if keyword in content:
return {
"messages": ({
"role": "assistant",
"content": "I cannot process requests containing inappropriate content."
}),
"jump_to": "end"
}
return None
```

This looks complicated, but it's straightforward. The `class ContentFilterMiddleware(AgentMiddleware)` line creates a new guardrail type. Think of it as designing a new security checkpoint.

The `__init__` The method sets up the guardrail when you create it. You pass in a list of banned words, and it stores them in lowercase for easy comparison.

The `@hook_config(can_jump_to=("end"))` The line is important. It tells the system that this guardrail can jump to the end of execution, meaning it can stop the agent immediately without processing anything.

The `before_agent` The method runs before the agent does anything. It receives the current state, which includes all messages in the conversation.

**The code checks if there are any messages, grabs the first one (the user's request), converts it to lowercase, and then loops through banned keywords. If it finds a match, it returns a message saying the request was blocked and jumps to the end. If no match, it returns None, which means "All clear, proceed normally."**

Using this custom guardrail:

```
agent = create_agent(
model="gpt-4o",
tools=(search_tool, calculator_tool),
middleware=(
ContentFilterMiddleware(banned_keywords=("hack", "exploit", "malware")),
),
)
```

Now, if someone asks, "How do I hack into a database?",

**the agent immediately responds with the block message. It never calls the AI model. It never uses tools. It just stops.**

## Smart Guards: Using AI to Check AI

Pattern matching catches obvious violations, but what about subtle problems? Requests that are technically allowed but contextually inappropriate? Content that follows all the rules but still feels wrong?

For this, you need AI-powered guardrails:

```
from langchain.chat_models import init_chat_model
class SafetyGuardrailMiddleware(AgentMiddleware):
def __init__(self):
super().__init__()
self.safety_model = init_chat_model("gpt-4o-mini")

 @hook_config(can_jump_to=("end"))
def after_agent(self, state: AgentState, runtime: Runtime):
if not state("messages"):
return None
last_message = state("messages")(-1)
safety_prompt = f"""Is this response safe and appropriate?
Answer only SAFE or UNSAFE.
Response: {last_message.content}"""
result = self.safety_model.invoke(({"role": "user", "content": safety_prompt}))
if "UNSAFE" in result.content:
last_message.content = "I cannot provide that response."
return None
```

This guardrail runs after the agent generates its final response. The `self.safety_model = init_chat_model("gpt-4o-mini")` The line creates a separate, smaller AI model specifically for safety checking. Using a smaller model keeps costs down since it runs on every response.

The `after_agent` The method grabs the last message (the agent's response) and asks the safety model to evaluate it. If the safety model says "UNSAFE", the guardrail replaces the entire response with a safe default message.

Why use a separate model?

Because you're essentially getting a second opinion. The main agent might generate something problematic. The safety model catches it before it reaches the user.

## Layering Protection: Defense in Depth

The most secure systems stack multiple guardrails. Each layer catches different types of problems. If one fails, others provide backup.

Here's a comprehensive setup:

```
agent = create_agent(
model="gpt-4o",
tools=(search_tool, send_email_tool),
middleware=(
ContentFilterMiddleware(banned_keywords=("hack", "exploit")),
PIIMiddleware("email", strategy="redact", apply_to_input=True),
PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
HumanInTheLoopMiddleware(interrupt_on={"send_email": True}),
SafetyGuardrailMiddleware(),
),
)
```

This creates five layers of protection. L

Layer one blocks obviously bad requests immediately using keyword matching. Fast and cheap.

Layer two strips email addresses from user input before the AI sees them.

Layer three masks credit card numbers in input.

Layer four requires human approval before sending emails. Layer five uses AI to evaluate the final response for safety issues.

Each guardrail runs at a specific point. Some run on input, some on output, some in the middle. Together, they create overlapping protection where bypassing all layers becomes nearly impossible.

## Practical Guardrails You Actually Need

Beyond the examples above, production systems typically need these additional protections:

**Rate limiting** stops agents from making thousands of API calls if something goes wrong. Set a limit like "Maximum 20 tool calls per conversation." If the agent hits this limit, it stops. This prevents runaway costs and infinite loops.

**Model call limits** prevent agents from calling the AI model endlessly without making progress. "Maximum 10 model calls per task." This catches agents stuck in unproductive loops.

**Tool-specific limits** restrict expensive operations. "Maximum 5 web searches per conversation" or "Maximum 3 database queries per request." This controls costs and prevents abuse of external services.

**Automatic fallbacks** provide backup when your primary AI model fails or becomes unavailable. "If GPT-4 fails, automatically try GPT-4 mini, then Claude." Your agent keeps working even during provider outages.

## Common Mistakes to Avoid

Organizations implementing guardrails often make predictable errors.

**Only checking inputs** is the most common mistake. You block bad requests but ignore bad responses. The AI might generate problematic content even from innocent requests. Always validate both input and output.

**Keyword-only protection** sounds good but fails fast. Users rephrase requests to bypass simple filters. "How do I gain unauthorized access" gets through a "hack" filter. Always combine pattern matching with AI-based evaluation.

**No testing with adversarial inputs** leaves gaps. Actually try to break your own guardrails. Get creative with phrasing. Try misspellings. Test edge cases. If you can bypass your guards, so can others.

**Vague error messages** frustrate users. "Request blocked" tells them nothing. Better: "Your request contained potentially sensitive information. Please remove email addresses and try again." Clear guidance helps legitimate users while still blocking attacks.

**Set and forget** guarantees obsolescence. Attack patterns evolve. New threats emerge. Review and update your guardrails based on real usage. Track what gets blocked and why.

## Measuring Success

How do you know if guardrails are working? Track these metrics:

**Block rate** shows how often guardrails trigger. Too low might mean insufficient coverage. Too high might mean overly aggressive blocking that hurts user experience.

**The false positive rate** measures legitimate requests incorrectly blocked. Keep this under 5%. Higher rates frustrate users and reduce trust.

**Response time impact** tracks the latency added by guardrails. Model-based checks add 200–500ms per evaluation. If your total response time exceeds 3 seconds, users notice.

**Cost per check** matters for AI-based guardrails. If safety checks cost more than the agent's actual work, your economics don't scale.

## The Real Cost of Not Having Guardrails

Let me be blunt about what happens without proper guardrails.

- **A leaked customer database costs millions in fines and settlements.**
- **A wrong financial transaction can trigger regulatory investigations.**
- **An inappropriate agent response going viral destroys the brand reputation built over the years.**

But there's also opportunity cost. **Without guardrails, you can't confidently deploy agents for high-value tasks. You're limited to low-risk use cases. Your competitors, with proper safety measures,s will move faster and capture more value.**

**Guardrails aren't overhead. Their infrastructure enables aggressive deployment of AI agents into production. They're the difference between "We're testing AI in a sandbox" and "Our AI agents handle millions of customer interactions daily."**

## Getting Started Tomorrow

If you're building AI agents today, start with these three guardrails:

**Add PII detection for any agent handling customer data. This prevents the most common and costly mistakes. Takes 10 minutes to implement.**

**Implement human-in-the-loop** for any destructive operations. Database deletions, financial transactions, mass emails. Get human eyes on these before execution.

Build a simple content filter for your specific domain. What words or phrases should never appear in requests or responses for your use case? Block those.

**These three guardrails catch 80% of problems with minimal implementation effort. Add more sophisticated protections as you learn what actually goes wrong in production.**

## The Bottom Line

AI agents represent a fundamental shift in how we build software. For the first time, we're deploying systems that make autonomous decisions and take actions without explicit programming for every scenario. That's incredibly powerful and incredibly risky.

**Guardrails aren't about limiting what AI can do. They're about safely unlocking what AI can do. They're the difference between "We can't trust AI with that" and "Our AI handles that automatically, and we sleep well at night."**

The companies winning with AI agents aren't necessarily the ones with the most advanced models. They're the ones who figured out how to deploy those models safely at scale. Guardrails are how you get there.

**Start simple. Add protection incrementally. Test aggressively. Monitor continuously. And remember that one prevented incident pays for years of guardrail development.**

The future belongs to AI agents. But only if we build them responsibly.

Published via Towards AI